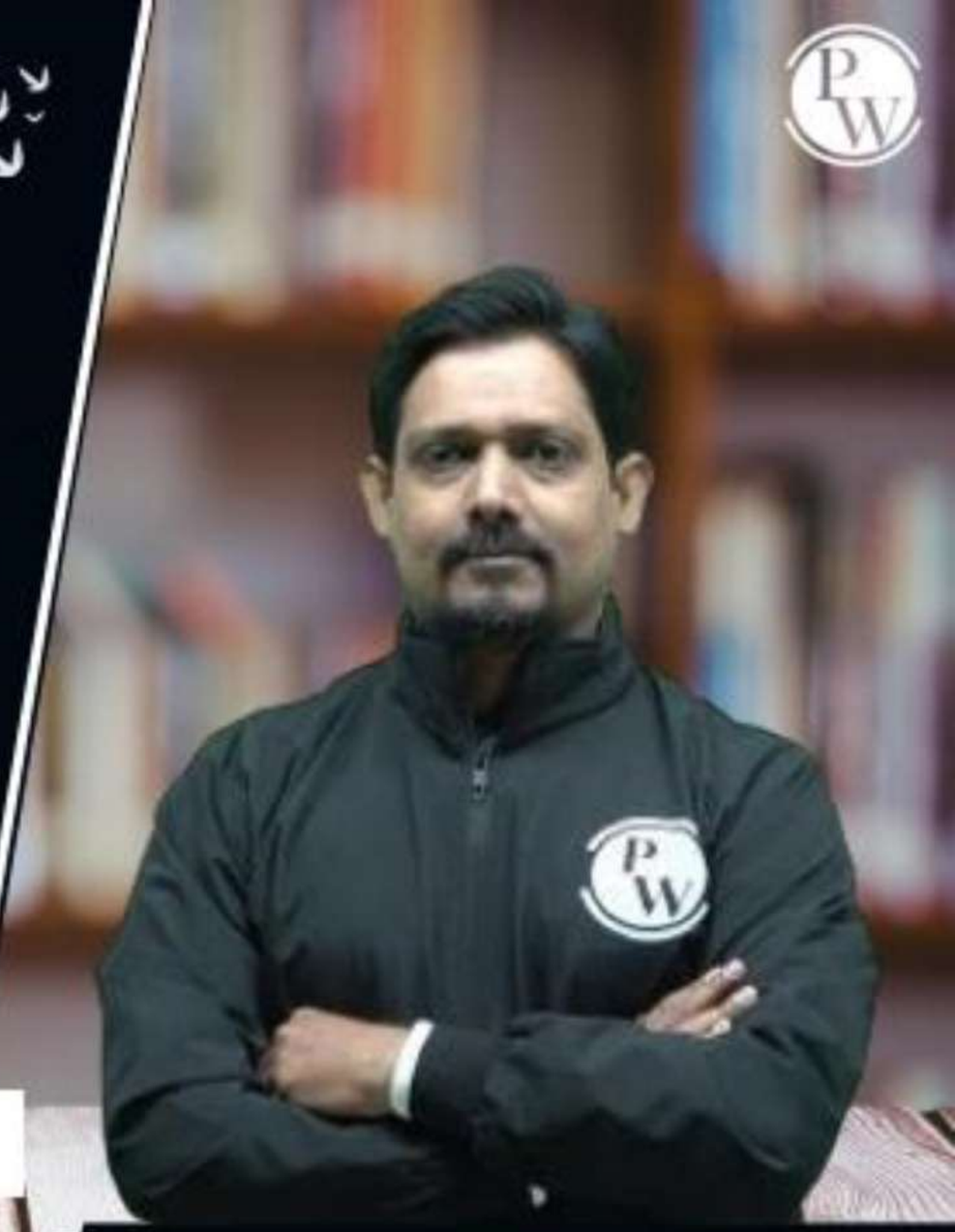
Computer Science & IT



Data Structure & Programming

Stack

Lecture No. 03



By- Abhishek Sir

Recap of Previous Lecture



- Topic
- Topic
- Topic
- Topic
- Topic

Stack as permutation generator

problem solving

Slide

Topics to be Covered



- Topic — Expression
- Topic — Infix
- Topic — prefix
- Topic — post fix
- Topic

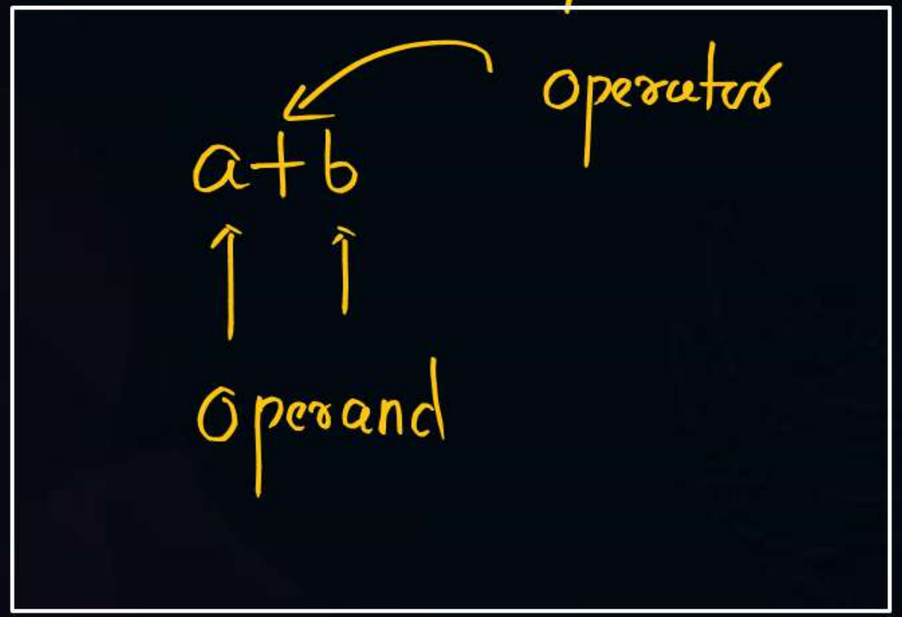
Slide



Topic : Expression In Computer

1 Operand — variable/constant

2 operator — Arthmetic operator



Slide



Topic : Expression In Computer

Infix expression a+b*c↑d — evaluate (precedence & Associativity)

repeated scanning ⟶ time complexity Increases

Evaluation using computer : we want Single scan evaluate the entire Expression that why prefix & postfix is used.

Slide



Topic : Expression In Computer

^
↑
exponention

( )			
−(uncay)			
↑, ^			Right Associahve
* %			Left
+, −			Left
=			Right associahve

2↑(3↑2)

2↑9=512

Slide



Topic : Expression In Computer

a+b operand$_1$ op operand$_2$

prefix : polish Notation : op operand$_1$ operand$_2$

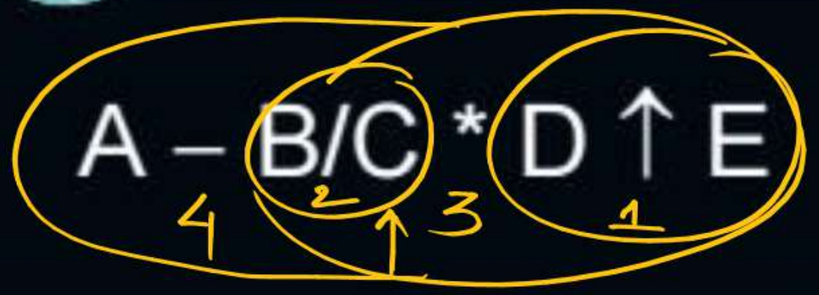
+ a b

postfix : Reverse polish : operand$_1$ operand$_2$ op

a b +

Slide

Topic : Expression In Computer



Operand

$= A - B/C * \underline{\uparrow DE}$

$= A - \underline{/BC} * \underline{\uparrow DE}$

$= A - \underline{* /BC \uparrow DE}$

$= - A * /BC \uparrow DE$

$- A - B/C * \underline{DE\uparrow}$

$= A - \underline{BC/} * \underline{DE\uparrow}$

$= A - \underline{BC/DE\uparrow *}$

$= ABC/DE\uparrow * -$

Slide

Topic : Expression In Computer

$A + B * C - D - E \uparrow F + G$

(3) (2) (4) (5) (1) (6)

Convert the above expression to prefix.

$= A + B*C - D - \uparrow EF + G$

$= A + *BC - D - \uparrow Ef + G$

$= +A*BC - D - \uparrow EF + G$

$= -+A*BCD - \uparrow EF + G$

$= --+A*BCD\uparrow EF+G$

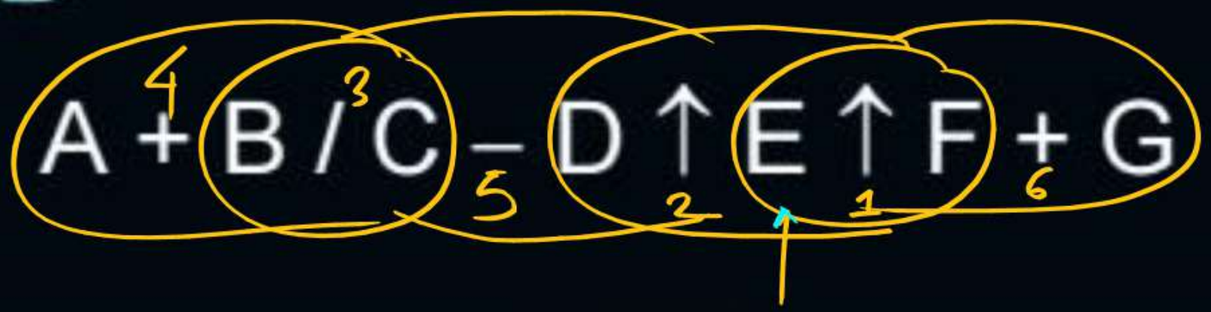
final Answer

$--+A*BCD\uparrow EFG$

( )	(1)
unary minus	(5)
$\uparrow$ ^	Right
* %	Left
+ -	Left
=	right

Slide

Topic : Expression In Computer



Convert the expression to postfix

$= A + B/C - D\uparrow \underline{EF\uparrow} + G$

$= A + B/C - \underline{DEF\uparrow\uparrow} + G$

$= A + \underline{BC/} - \underline{DEF\uparrow\uparrow} + G$

$= \underline{ABC/+} - \underline{DEF\uparrow\uparrow} + G$

$= \underline{ABC/+DEF\uparrow\uparrow-} + G \Rightarrow$

ABC/+DEF↑↑−G+

$a+b+c$ → $+ab+c$ → **+ + abc**

$a+b+c$ → $ab+ +c$

ab+c+

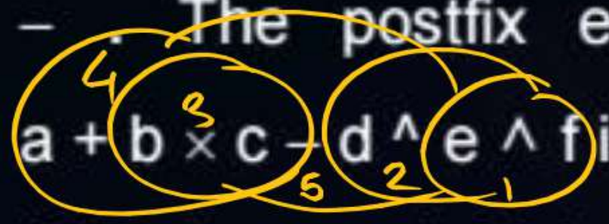
Slide



Topic : Expression In Computer

PYQ - 2004

Assume that the operators + , − , ×, are left associative and ^ is right associative. The order of precedence (from highest to lowest) is ^, ×, +, −. The postfix expression corresponding to the infix expression a + b × c − d ^ e ^ f is



(A) abc × + def ^ ^ −

(B) abc × + de ^f ^ −

(C) ab + c × d − e ^ f ^

(D) − + a × bc ^ ^ def

a + b × c − d ^ e ^ f

a + b × c − d ^ ef ^

a + b × c − def ^ ^

a + bcx − def ^ ^

abcx + − def ^ ^ =

abcx + def ^ ^ −

Slide



Topic : Expression In Computer

The **prefix** expression for the infix expression

a = – b +c * d/e + f ↑ g ↑ h – i * j

(a) = a – + + – b/*cde ↑f ↑gh*ij

(b) = a – + + – /*bcde ↑f ↑gh*ij

(c) = a + – + – b/*cde ↑f ↑gh*ij

(d) = a – + – – b/*cde ↑f ↑ah*ii

Slide



Topic : Expression In Computer

1. Scan the input left to Right
2. output printed left to right
3. if input is an operand then print the operand.
4. if input is an operator & stack is empty then push the operator
5. if input is an operator & having higher precedance than operator on top of stack then push New operator in stack.

HPU

6. if input is an operator & having lower precedence than operator on top of stack then we pop operators till top of stack having higher precedence. and then push the new operator.

LPOPU

s

Slide

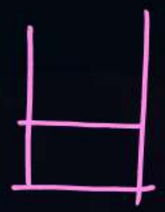
Topic : Expression In Computer

Infix → postfix using stack (operator stack)

a+(b*c) → a+bc* → abc*+

① Input: a

output: a



② Input: +

output: a



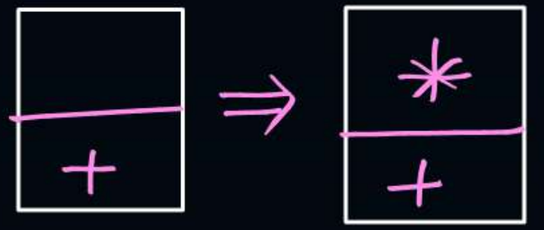
③ Input: b

output: ab



④ Input: *

output: ab



⑤ Input: c

output: abc

⑥ end of input: pop everything & print

output: abc*+

Slide

Topic : Expression In Computer

Infix to postfix

a↑b*c-d

① Input : a

output : a

② Input : ↑

output : a

③ Input : b

output : ab

stack



stack



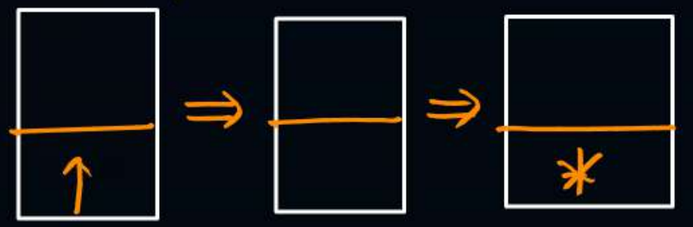
stack



④ Input : *

output : ab

stack pop↑ and push*



ab↑

⑤ Input : c

output : ab↑c

stack

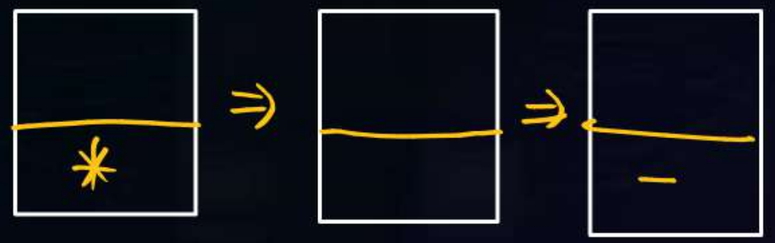


⑥ Input : -

output : ab↑c

ab↑c*

⑦ Input : d output : ab↑c*d



⑧ End of input

ab↑c*d-



Slide



Topic : Expression In Computer

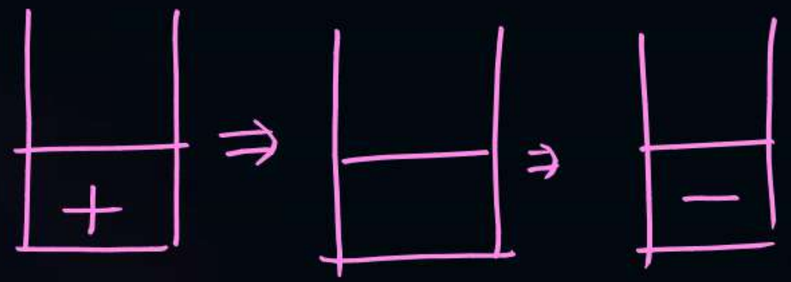
Same precedence

a + b − c

Input : a

output : ab+

pop+ & push−



Left Associative :- LPOPU-S

− lower precedence than +

Left Associative.

⇒ Input : c

output ab+c

⇒ End of input : ab+c−



Slide

Topic : Expression In Computer

$a = b = c$
$\quad\quad 1 \quad 2$

⇒ Input : a
output : a

⇒ Input : $=_1$
output : a



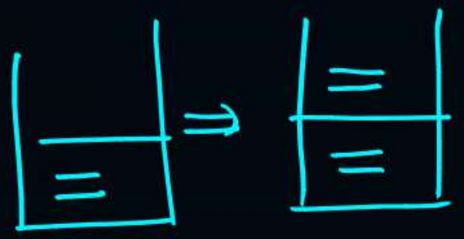
⇒ Input : b
output : ab

⇒ Input : $=_2$
output : ab

⇒ Input :
output : abc

⇒ end of input

abc = =



Right Associativity

$=_2$ Higher precedence

HPU

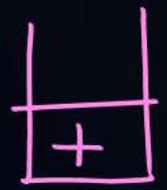
Slide

Topic : Expression In Computer

$a+b\uparrow c-d$

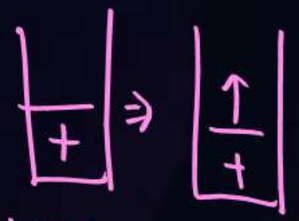
① Input: a
output: a

② Input: +
output: a



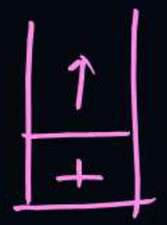
③ Input: b
output: ab

④ Input: ↑
output: ab



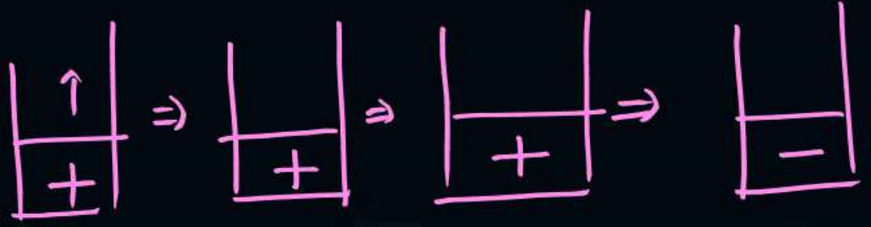
HPU

⑤ Input: c
output: abc



⑤ Input: −
output: − abc

− Lower − lower than +



$abc\uparrow +$

⑥ Input: d
output: abc↑+d



⑦ end of input

abc↑+d−

Slide



Topic : Expression In Computer

1. Reverse the expression
2. Convert into postfix

   Consider original expression

   precedence & Associativity
3. Reverse the postfix expression

a+b-c

-+abc

c-b+a

Input: | c | - | b | + | END |

output: cba+-



Reverse

-+abc

Slide



Topic : Expression In Computer

PYQ – 2024

Consider the operator precedence and associativity rules for the *integer* arithmetic operators given in the table below.

Operator	Precedence	Associativity
+	Highest	Left
-	High	Right
*	Medium	Right
/	Low	Right

The value of the expression $3 + 1 + 5 * 2/7 + 2 - 4 - 7 - 6/2$ as per the above rules is _____.

Slide

ator precedence and associativity rules for the *integer* arithmetic
the table below.

Operator	Precedence	Associativity
+	Highest	Left
-	High	Right
*	Medium	Right
/	Low	Right

xpression $3+1+5*2/7+2-4-7-6/2$ as per the above rules

$3+1+5*2/7+2-4-$

$9*2/9-4-7-6/$

$9*2/9-4-1/2$

$9*2/9-3/2$

$9*2/6/2$

$= 18/6/2 = 18/3 =$



Topic : Question

PYQ-2016

The attribute of three arithmetic operators in some programming language are given below.

OPERATOR	PRECEDENCE	ASSOCIATIVITY	ARITY
+	High	Left	Binary
−	Medium	Right	Binary
*	Low	Left	Binary

The value of the expression $2 - 5 + 1 - 7 * 3$ in this language is _____.

$2 - 5 + 1 - 7 * 3 \quad = 2 - (6 - 7) * 3$

$2 - 6 - 7 * 3 \quad = 2 - (-1) * 3 = 3 * 3 = 9$

Slide



Topic : Question

2003

Let s be a stack of size $n \geq 1$. Starting with the empty stack, suppose we push the first n natural numbers in sequence, and then perform n pop operations. Assume that *Push* and *Pop* operations take X seconds each, and Y seconds elapse between the end of one such stack operation and the start of the next operation. For $m \geq 1$, define the stack-life of m as the time elapsed from the end of Push (m) to the start of the pop operation that removes m form S. The average stack life of an element of this stack is

Homework

(A) $n(X + Y)$

(B) $3Y + 2X$

(C) $n(X + Y) - X$

(D) $Y + 2X$



2 mins Summary

Topic — polish Notation

Topic — Reverse polish

Topic — Infix → post-fix using stack

Topic — Infix to prefix using Reversal Method

Topic

Slide

THANK - YOU